

Chapter 2: Memory Hierarchy Design – Part 2

Introduction (Section 2.1, Appendix B)

Caches

Review of basics (Section 2.1, Appendix B)

Advanced methods (Section 2.3)

Main Memory

Virtual Memory

Fundamental Cache Parameters

Cache Size

How large should the cache be?

Block Size

What is the smallest unit represented in the cache?

Associativity

How many entries must be searched for a given address?

Cache Size

Cache size is the total capacity of the cache

Bigger caches exploit temporal locality better than smaller caches

But are *not always* better Why?

Cache Size**

Cache size is the total capacity of the cache

Bigger caches exploit temporal locality better than smaller caches

But are *not always* better

Too large a cache size

Smaller means faster $\Rightarrow$ bigger means slower

Access time may degrade critical path

Too small a cache size

Don't exploit temporal locality well

Useful data is prematurely replaced

Block Size

Block (line) size is the data size that is both

- (a) associated with an address tag, and

- (b) transferred to/from memory

Advanced caches allow different (a) & (b)

Problem with too small blocks

Problem with large blocks

Block Size**

Block (line) size is the data size that is both

- (a) associated with an address tag, and

- (b) transferred to/from memory

Advanced caches allow different (a) & (b)

Too small blocks

- Don't exploit spatial locality well

- Don't amortize memory access time well

- Have inordinate address tag overhead

Too large blocks cause

Block Size**

Block (line) size is the data size that is both

- (a) associated with an address tag, and
- (b) transferred to/from memory

Advanced caches allow different (a) & (b)

Too small blocks

- Don't exploit spatial locality well

- Don't amortize memory access time well

- Have inordinate address tag overhead

Too large blocks cause

- Unused data to be transferred

- Useful data to be prematurely replaced

Set Associativity

Partition cache block frames & memory blocks in equivalence classes (usually set bits selection)

Number of sets, s , is the number of classes

Associativity (set size), n , is the number of block frames per class

Number of block frames in the cache is $s \times n$

Cache Lookup (assuming read hit)

- Select set

- Associatively compare stored tags to incoming tag

- Route data to processor

Associativity, cont.

Typical values for associativity

1 -- direct-mapped

$n = 2, 4, 8, 16$ -- n -way set-associative

All blocks – fully-associative

Larger associativity

Smaller associativity

Associativity, cont. **

Typical values for associativity

- 1 -- direct-mapped

- $n = 2, 4, 8, 16$ -- n -way set-associative

- All blocks – fully-associative

Larger associativities

- Lower miss ratios

- Less variance

Smaller associativities

- Lower cost

- Faster access (hit) time (perhaps)

Advanced Cache Optimizations

Reducing the Hit Time

Increasing Cache Bandwidth

Reducing the Miss Penalty

Reducing the Miss Rate

Reducing the Miss Penalty or Miss Rate via Parallelism

Advanced Cache Optimizations

Reducing the Hit Time

- ① Small and simple first-level caches

Increasing Cache Bandwidth

Reducing the Miss Penalty

Reducing the Miss Rate

Reducing the Miss Penalty or Miss Rate via Parallelism

Small and Simple First-Level Caches

Critical Timing Path

1. Use the index portion of the address
2. Compare the red tag value to the address
3. Choose the correct data item

Limited Size for First-Level Caches

- Fast clock cycle and power limitations

Direct-mapped caches can overlap tag compare and transmission of data

- Parallelism for reducing hit time

Lower Associativity Reduce Hit Time and Power

- Fewer cache lines are accessed
- Simple cache structure
- Less times of comparing tags

L1 Size and Associativity

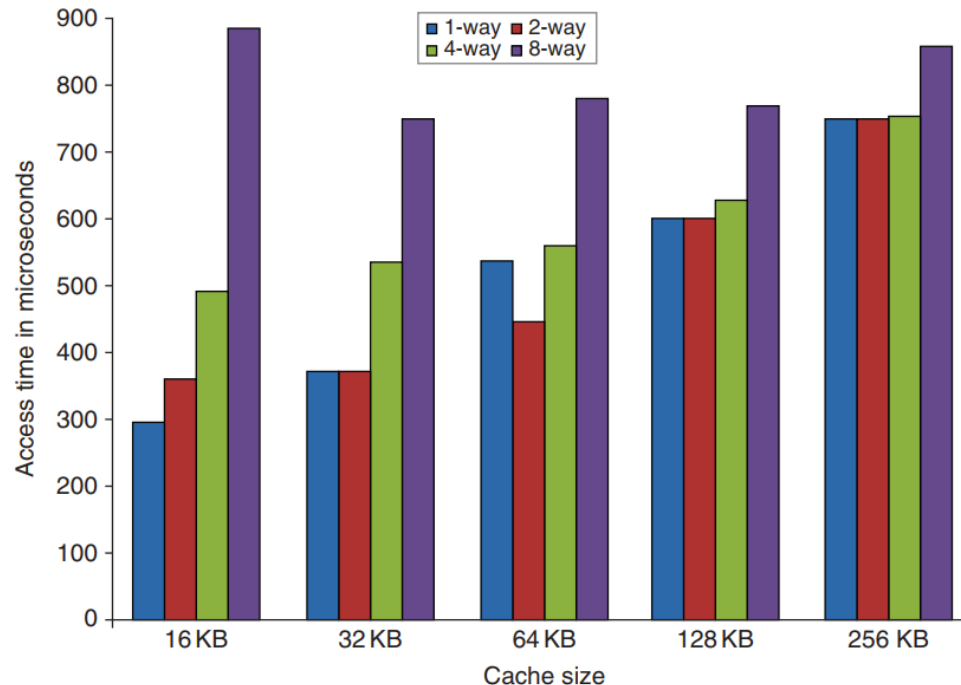


Figure 2.3 Access times generally increase as cache size and associativity are increased. These data come from the CACTI model 6.5 by Tarjan, Thoziyoor, and Jouppi [2005]. The data assume a 40 nm feature size (which is between the technology used in Intel's fastest and second fastest versions of the i7 and the same as the technology used in the fastest ARM embedded processors), a single bank, and 64-byte blocks. The assumptions about cache layout and the complex trade-offs between interconnect delays (that depend on the size of a cache block being accessed) and the cost of tag checks and multiplexing lead to results that are occasionally surprising, such as the lower access time for a 64 KB with two-way set associativity versus direct mapping. Similarly, the results with eight-way set associativity generate unusual behavior as cache size is increased. Since such observations are highly dependent on technology and detailed design assumptions, tools such as CACTI serve to reduce the search space rather than precision analysis of the trade-offs.

L1 Size and Associativity

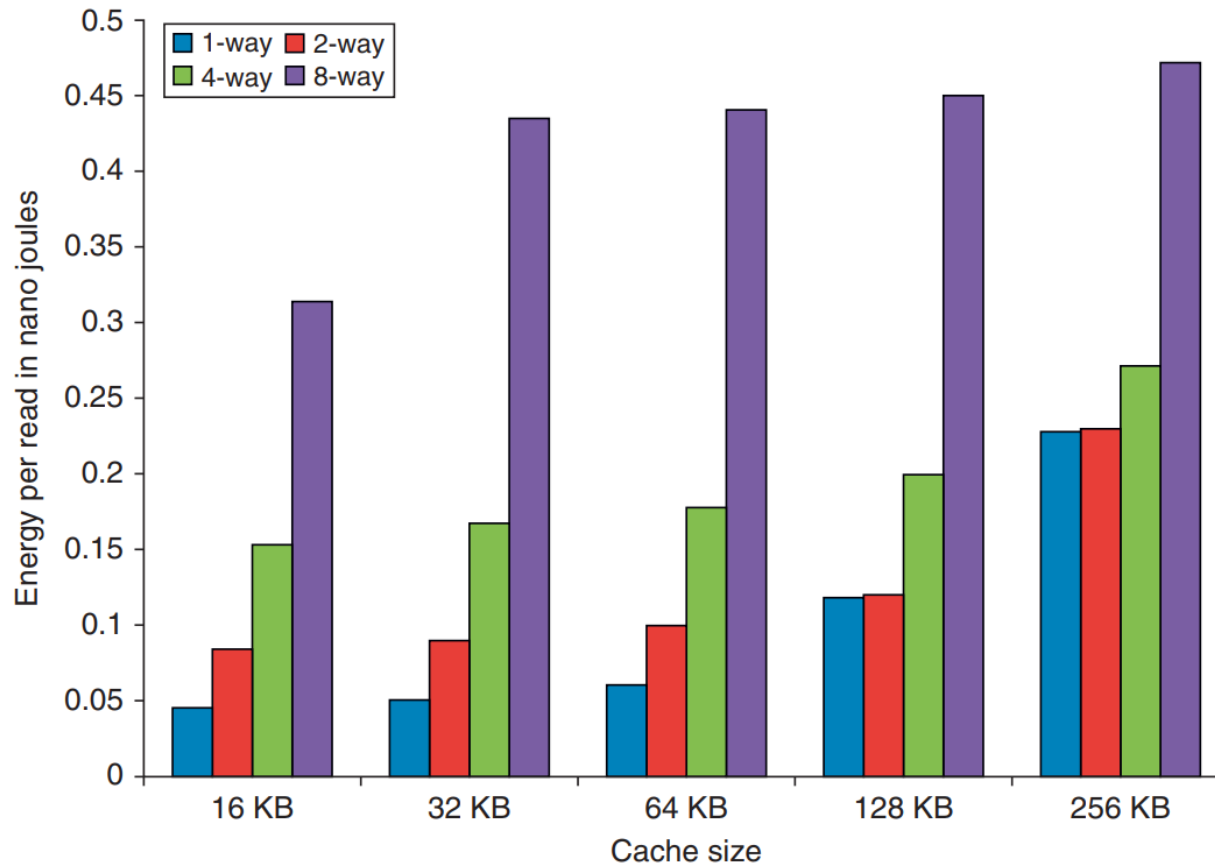


Figure 2.4 Energy consumption per read increases as cache size and associativity are increased. As in the previous figure, CACTI is used for the modeling with the same technology parameters. The large penalty for eight-way set associative caches is due to the cost of reading out eight tags and the corresponding data in parallel.

Advanced Cache Optimizations

Reducing the Hit Time

- ① Small and simple first-level caches
- ② Way prediction

Increasing Cache Bandwidth

Reducing the Miss Penalty

Reducing the Miss Rate

Reducing the Miss Penalty or Miss Rate via Parallelism

Way Prediction

Keep extra bits in cache to predict the “way” of the next access

Access predicted way first

If miss, access other ways like in set associative caches

Fast hit when prediction is correct

Way Prediction

Reduce Hit Time

- Reduce conflict misses
- Yet maintain the hit speed of direct-mapped caches
- A latency of one extra clock cycle on a way misprediction
- Accuracy of Prediction
 - 90%+ for two-way, 80%+ for four-way
 - I-caches are better than D-caches
- MIPS R10000 (first used), ARM Cortex-A8 (four-way)

Reduce Power Consumption (Extended Form)

- Be called way selection
- Save power when the way prediction is correct
- But add significant time on a way misprediction

Advanced Cache Optimizations

Reducing the Hit Time

- ① Small and simple first-level caches
- ② Way prediction

Increasing Cache Bandwidth

- ③ Pipelined Cache Access

Reducing the Miss Penalty

Reducing the Miss Rate

Reducing the Miss Penalty or Miss Rate via Parallelism

Pipelined Cache Access

Pipelining

- Old: cache access = 1 cycle
- New: 1 cycle caches would slow the whole processor
- Pipeline: cache hit may take 4 cycles (increase mispredicted branches penalty and clock cycles between issuing the load and using the data)

Pipelining Cache

- Faster clock cycle time
- Higher bandwidth, easier to incorporate high degrees of associativity
- Slower hit time

The pipeline for the i-cache access

- Pentium, 1 clock cycle
- Pentium pro - Pentium III, 2 clock cycle
- Pentium IV - Core i7, 4 clock cycle

Pipelined Cache Access

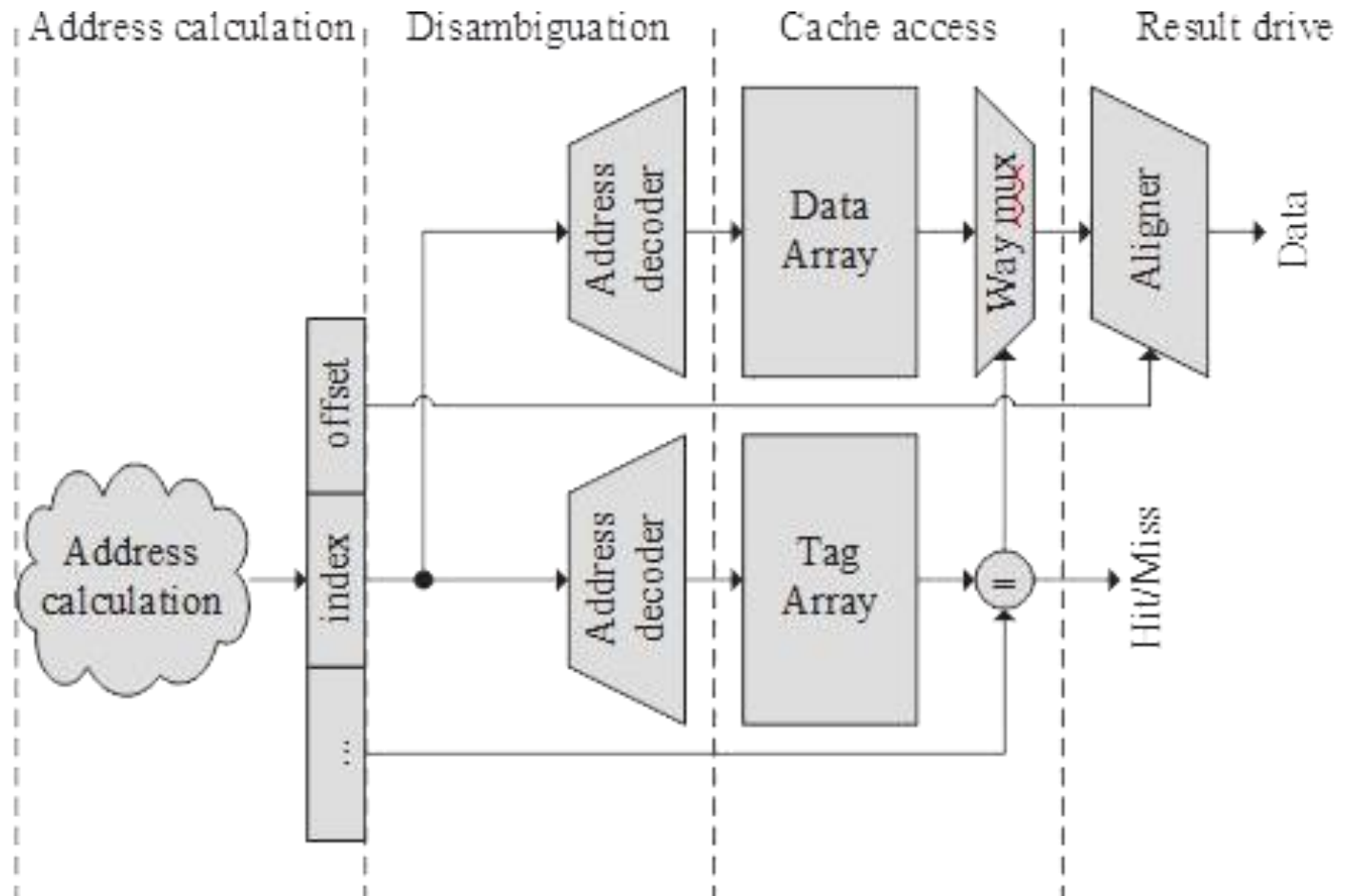


FIGURE 2.2: Parallel tag and data array access pipeline.

Pipelined Cache Access

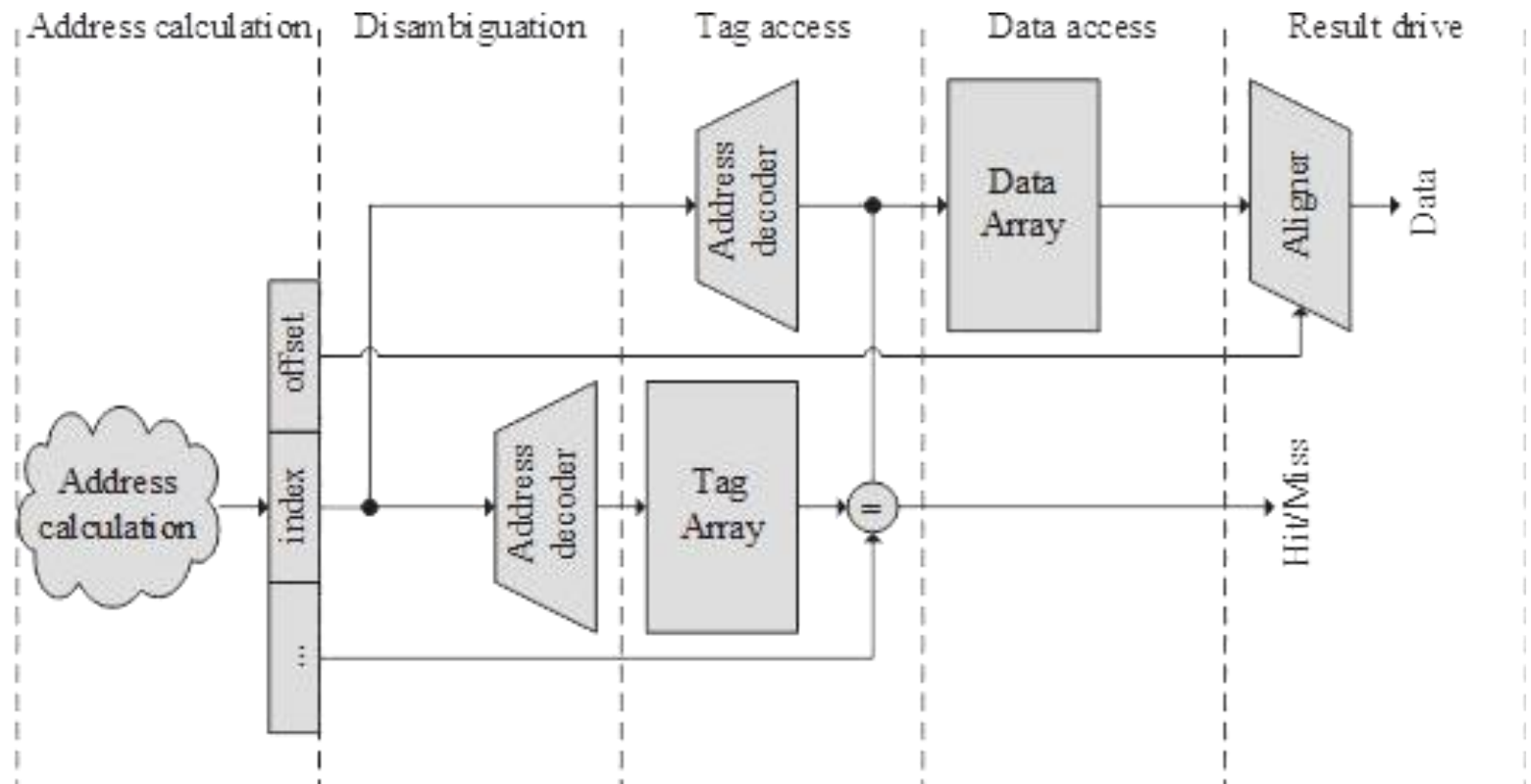


FIGURE 2.4: Serial tag and data array access pipeline.

Advanced Cache Optimizations

Reducing the Hit Time

- ① Small and simple first-level caches
- ② Way prediction

Increasing Cache Bandwidth

- ③ Pipelined access
- ④ Multibanked caches

Reducing the Miss Penalty

Reducing the Miss Rate

Reducing the Miss Penalty or Miss Rate via Parallelism

Multibanked Caches

Divide Cache into Independent Banks, Support Simultaneous Accesses

- Increase cache bandwidth
- Reduce power consumption
- ARM Cortex-A8 supports 1-4 banks for L2
- Intel i7 supports 4 banks for L1 and 8 banks for L2

Block based Interleaving Allows Multiple Accesses Per Cycle

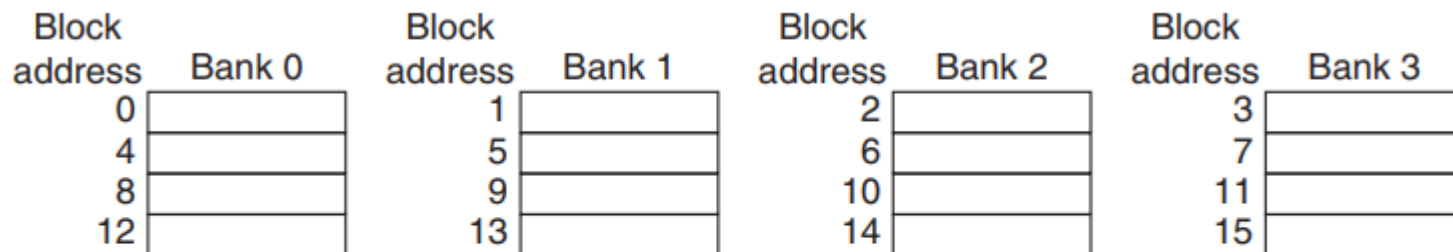


Figure 2.6 Four-way interleaved cache banks using block addressing. Assuming 64 bytes per blocks, each of these addresses would be multiplied by 64 to get byte addressing.

Advanced Cache Optimizations

Reducing the Hit Time

- ① Small and simple first-level caches
- ② Way prediction

Increasing Cache Bandwidth

- ③ Pipelined access
- ④ Multibanked caches
- ⑤ Nonblocking caches (Lookup-Free Caches)

Reducing the Miss Penalty

Reducing the Miss Rate

Reducing the Miss Penalty or Miss Rate via Parallelism

Lockup-Free Caches

Normal cache stalls while a miss is pending

Lockup-Free Caches

- (a) Handle hits while first miss is pending
- (b) Handle hits & misses until K misses are pending

Potential benefit

- (a) Overlap misses with useful work & hits
- (b) Also overlap misses with each other

Only makes sense if

Lockup-Free Caches**

Normal cache stalls while a miss is pending

Lockup-Free Caches

- (a) Handle hits while first miss is pending
- (b) Handle hits & misses until K misses are pending

Potential benefit

- (a) Overlap misses with useful work & hits
- (b) Also overlap misses with each other
- (c) Increase cache bandwidth
- (d) Reduce miss penalty

Only makes sense if processor

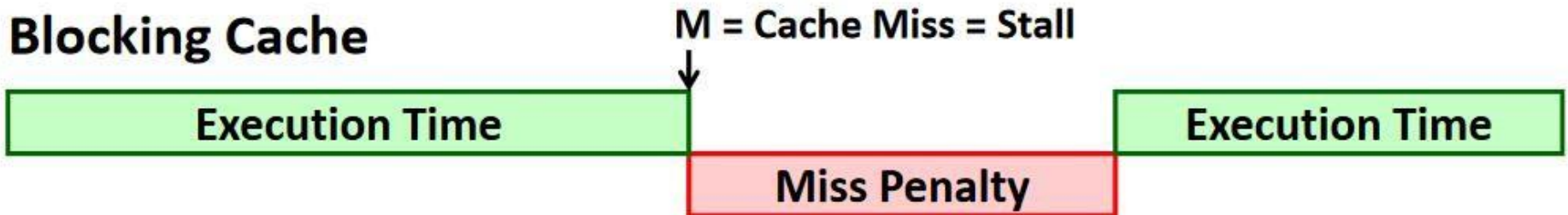
Handles pending references correctly

Often can do useful work with references pending (Tomasulo, etc.)

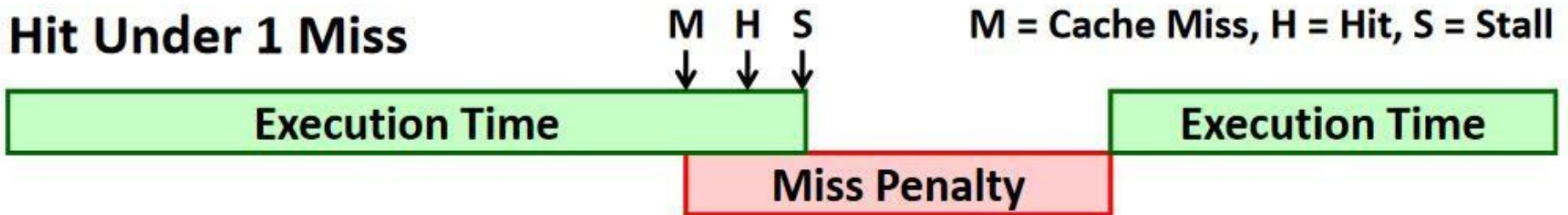
Has misses that can be overlapped (for (b))

Lockup-Free Caches Timelines

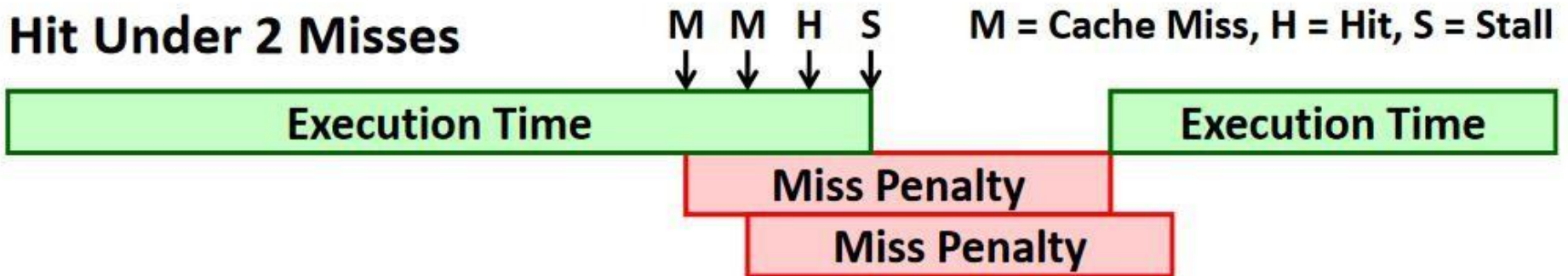
Blocking Cache



Hit Under 1 Miss



Hit Under 2 Misses



Lockup-Free Caches, cont.

Key implementation problems

- (1) Handling reads to pending miss
- (2) Handling writes to pending miss
- (3) Keep multiple requests straight

MSHRs -- miss status holding registers

What state do we need in MSHR?

Lockup-Free Caches, cont. **

Key implementation problems

- (1) Handling reads to pending miss
- (2) Handling writes to pending miss
- (3) Keep multiple requests straight

MSHRs -- miss status holding registers

For every MSHR – Valid bit

Block request address

For every word – Destination register?

Valid bit

Format (byte load, etc.)

Comparator (for later miss requests)

Limitation?

*Lockup-Free Caches, cont.***

Key implementation problems

- (1) Handling reads to pending miss
- (2) Handling writes to pending miss
- (3) Keep multiple requests straight

MSHRs -- miss status holding registers

For every MSHR – Valid bit

Block request address

For every word – Destination register?

Valid bit

Format (byte load, etc.)

Comparator (for later miss requests)

Limitation: Must block on next access to same word

Advanced Cache Optimizations

Reducing the Hit Time

- ① Small and simple first-level caches
- ② Way prediction

Increasing Cache Bandwidth

- ③ Pipelined access
- ④ Multibanked caches
- ⑤ Nonblocking caches (Lookup-Free Caches)

Reducing the Miss Penalty

- ⑥ Critical word first and early restart

Reducing the Miss Rate

Reducing the Miss Penalty or Miss Rate via Parallelism

Critical Word First, Early Restart

Critical Word First

- a) Request the missed word first from memory
- b) Send it to the processor as soon as it arrives
- c) Let the processor continue execution while filling the rest of the words in the block

Early Restart

- a) Fetch the words in normal order
- b) Send it to the processor as soon as the requested word of the block arrives
- c) Let the processor continue execution

Only Benefit Designs with Large Cache Blocks

Advanced Cache Optimizations

Reducing the Hit Time

- ① Small and simple first-level caches
- ② Way prediction

Increasing Cache Bandwidth

- ③ Pipelined access
- ④ Multibanked caches
- ⑤ Nonblocking caches (Lookup-Free Caches)

Reducing the Miss Penalty

- ⑥ Critical word first and early restart
- ⑦ Merging write buffer

Reducing the Miss Rate

Reducing the Miss Penalty or Miss Rate via Parallelism

Merging Write Buffer

Write Merging

- a) If the buffer contains other modified blocks, the addresses can be checked to see if the address of the new data matches the address of a valid write buffer entry.
- b) If so, the new data are combined with that entry.

Multiword writes are faster than writes performed one word at a time

Reduces stalls due to the write buffer being full

Write address	V		V		V		V
100	1	Mem[100]	0		0		0
108	1	Mem[108]	0		0		0
116	1	Mem[116]	0		0		0
124	1	Mem[124]	0		0		0

No write buffering

Write address	V		V		V		V	
100	1	Mem[100]	1	Mem[108]	1	Mem[116]	1	Mem[124]
	0		0		0		0	
	0		0		0		0	
	0		0		0		0	

Write buffering

Advanced Cache Optimizations

Reducing the Hit Time

- ① Small and simple first-level caches
- ② Way prediction

Increasing Cache Bandwidth

- ③ Pipelined access
- ④ Multibanked caches
- ⑤ Nonblocking caches (Lookup-Free Caches)

Reducing the Miss Penalty

- ⑥ Critical word first and early restart
- ⑦ Merging write buffer

Reducing the Miss Rate

- ⑧ Compiler optimizations

Reducing the Miss Penalty or Miss Rate via Parallelism

Compiler Optimizations

Reduces Miss Rates Without Any Hardware Changes

- Improvements in Instruction Misses
- Improvements in Data Misses

Instructions

- Reorder procedures in memory so as to reduce conflict misses
- Profiling to look at conflicts (using tools)

Data

- **Merging Arrays:** improve spatial locality by single array of compound elements vs. 2 arrays
- **Loop Interchange:** change nesting of loops to access data in order stored in memory
- **Loop Fusion:** Combine 2 independent loops that have same looping and some variables overlap
- **Blocking:** Improve temporal locality by accessing “blocks” of data repeatedly vs. going down whole columns or rows

Merging Arrays Example

```
/* Before: 2 sequential arrays */
```

```
int val[SIZE];
```

```
int key[SIZE];
```

```
/* After: 1 array of structures */
```

```
struct merge {
```

```
    int val;
```

```
    int key;
```

```
};
```

```
struct merge merged_array[SIZE];
```

Reducing conflicts between val & key

Improve spatial locality

Loop Interchange Example

```
/* Before */
```

```
for (k = 0; k < 100; k = k+1)
```

```
    for (j = 0; j < 100; j = j+1)
```

```
        for (i = 0; i < 5000; i = i+1)
```

```
            x[i][j] = 2 * x[i][j];
```

```
/* After */
```

```
for (k = 0; k < 100; k = k+1)
```

```
    for (i = 0; i < 5000; i = i+1)
```

```
        for (j = 0; j < 100; j = j+1)
```

```
            x[i][j] = 2 * x[i][j];
```

**Sequential accesses instead of striding through memory every 100 words;
Improved spatial locality**

Loop Fusion Example

```
/* Before */
```

```
for (i = 0; i < N; i = i+1)
```

```
    for (j = 0; j < N; j = j+1)
```

```
        a[i][j] = 1/b[i][j] * c[i][j];
```

```
for (i = 0; i < N; i = i+1)
```

```
    for (j = 0; j < N; j = j+1)
```

```
        d[i][j] = a[i][j] + c[i][j];
```

```
/* After */
```

```
for (i = 0; i < N; i = i+1)
```

```
    for (j = 0; j < N; j = j+1)
```

```
    {      a[i][j] = 1/b[i][j] * c[i][j];
```

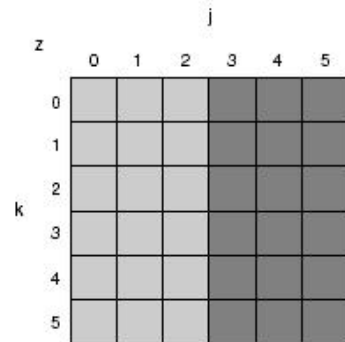
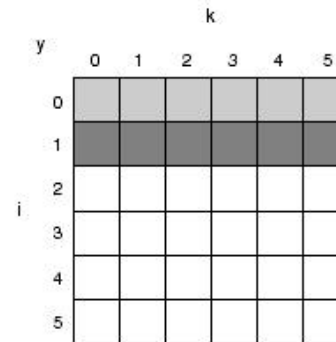
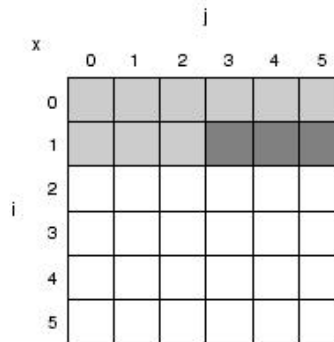
```
        d[i][j] = a[i][j] + c[i][j]; }
```

**2 misses per access to a & c vs. one miss per access;
Improve spatial locality**

Blocking Example

```
/* Before */
```

```
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1)
        {r = 0;
          for (k = 0; k < N; k = k+1) {
            r = r + y[i][k]*z[k][j];
          }
          x[i][j] = r;
        };
```



Two Inner Loops:

- Read all $N \times N$ elements of $z[]$

- Read N elements of 1 row of $y[]$ repeatedly

- Write N elements of 1 row of $x[]$

Capacity Misses a function of N & Cache Size:

$2N^3 + N^2 \Rightarrow$ (assuming no conflict; otherwise ...)

Idea: compute on $B \times B$ submatrix that fits

Blocking Example

```

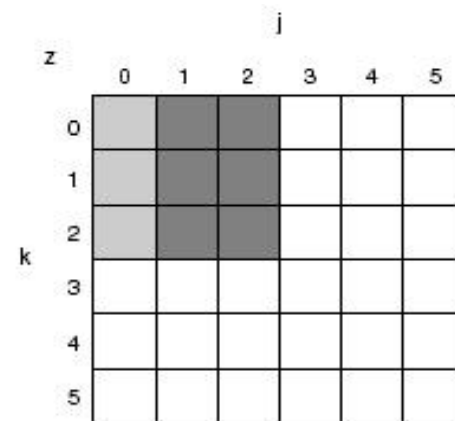
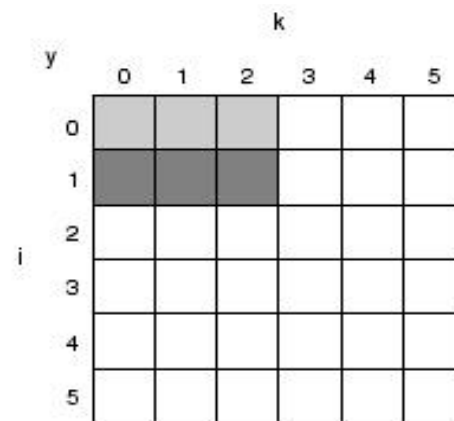
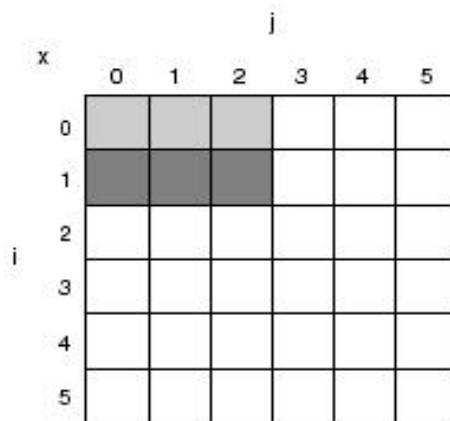
/* After */
for (jj = 0; jj < N; jj = jj+B)
for (kk = 0; kk < N; kk = kk+B)
for (i = 0; i < N; i = i+1)
    for (j = jj; j < min(jj+B-1,N); j = j+1)
        {r = 0;
          for (k = kk; k < min(kk+B-1,N); k = k+1) {
              r = r + y[i][k]*z[k][j];};
          x[i][j] = x[i][j] + r;
        };

```

B called Blocking Factor

Capacity Misses from $2N^3 + N^2$ to $2N^3/B + N^2$

Conflict Misses Too?



Advanced Cache Optimizations

Reducing the Hit Time

- ① Small and simple first-level caches
- ② Way prediction

Increasing Cache Bandwidth

- ③ Pipelined access
- ④ Multibanked caches
- ⑤ Nonblocking caches (Lookup-Free Caches)

Reducing the Miss Penalty

- ⑥ Critical word first and early restart
- ⑦ Merging write buffer

Reducing the Miss Rate

- ⑧ Compiler optimizations

Reducing the Miss Penalty or Miss Rate via Parallelism

- ⑨ Hardware prefetching of Instructions and Data

Prefetching

Prefetch instructions/data before processor requests them

Even ``demand fetching" prefetches other words in the referenced block

Prefetching is useless unless a prefetch ``costs" less than demand miss

Prefetching relies on having extra memory bandwidth that can be used without penalty

Prefetches should

???

Prefetching**

Prefetch instructions/data before processor requests them

Even ``demand fetching" prefetches other words in the referenced block

Prefetching is useless unless a prefetch ``costs" less than demand miss

Prefetching relies on having extra memory bandwidth that can be used without penalty

Prefetches should

- (a) Always get data back before it is referenced
 - (b) Never get data not used
 - (c) Never prematurely replace other data
 - (d) Never interfere with other cache activity
- Item (a) conflicts with (b), (c) and (d)

Prefetching Policy

Policy

What to prefetch?

When to prefetch?

Simplest Policy

?

Enhancements

Prefetching Policy**

Policy

What to prefetch?

When to prefetch?

Simplest Policy

One block (spatially) ahead on every access

Enhancements

Prefetching Policy **

Policy

- What to prefetch?

- When to prefetch?

Simplest Policy

- One block (spatially) ahead on every access

Enhancements

- On every miss

 - Because hard to determine on every reference whether block to be prefetched is already in cache

- Detect stride and prefetch on stride

- Prefetch into prefetch buffer

- Prefetch more than 1 block (for instruction caches, when block small)

Advanced Cache Optimizations

Reducing the Hit Time

- ① Small and simple first-level caches
- ② Way prediction

Increasing Cache Bandwidth

- ③ Pipelined access
- ④ Multibanked caches
- ⑤ Nonblocking caches (Lookup-Free Caches)

Reducing the Miss Penalty

- ⑥ Critical word first and early restart
- ⑦ Merging write buffer

Reducing the Miss Rate

- ⑧ Compiler optimizations

Reducing the Miss Penalty or Miss Rate via Parallelism

- ⑨ Hardware prefetching of Instructions and Data
- ⑩ Compiler-Controlled prefetching

Software Prefetching

Use compiler to

Prefetch early

E.g., one loop iteration ahead

Prefetch accurately

Software Prefetching Example

```
for (i = 0; i < N-1; i++) {  
    ... = A[i]  
    /* computation */  
}
```

Assume each iteration takes 10 cycles with a hit,
memory latency is 100 cycles

*Software Prefetching Example***

```
for (i = 0; i < N-1; i++) {  
    ... = A[i]  
    /* computation */  
}
```

Assume each iteration takes 10 cycles with a hit,
memory latency is 100 cycles

```
for (i = 0; i < N-1; i++) {  
    prefetch(A[i+10])  
    ... = A[i]  
    /* computation */  
}
```

*Software Prefetching Example***

```
for (i = 0; i < N-1; i++) {  
    ... = A[i]  
    /* computation */  
}
```

Assume each iteration takes 10 cycles with a hit,
memory latency is 100 cycles

```
for (i = 0; i < N-1; i++) {  
    prefetch(A[i+10])  
    ... = A[i]  
    /* computation */  
}
```

This is good with one word cache blocks

Software Prefetching Example

```
for (i = 0; i < N-1; i++) {  
    ... = A[i]  
    /* computation */  
}
```

Assume each iteration takes 10 cycles with a hit,
memory latency is 100 cycles, **cache block is two words**

Changes?

```
for (i = 0; i < N-1; i++) {  
    prefetch(A[i+10])  
    ... = A[i]  
    /* computation */  
}
```

Software Prefetching Example**

```
for (i = 0; i < N-1; i++) {  
    ... = A[i]  
    /* computation */  
}
```

Assume each iteration takes 10 cycles with a hit,
memory latency is 100 cycles, **cache block is two words**

```
for (i = 0; i < N-1; i+=2) {  
    prefetch(A[i+10])  
    ... = A[i]  
    ... = A[i+1]  
    /* computation for two iterations */  
}
```

A1: Victim Cache

Add a small fully associative cache next to main cache

On a miss in main cache

A1: Victim Cache**

Add a small fully associative cache next to main cache

On a miss in main cache

Search in victim cache

Put any replaced data in victim cache

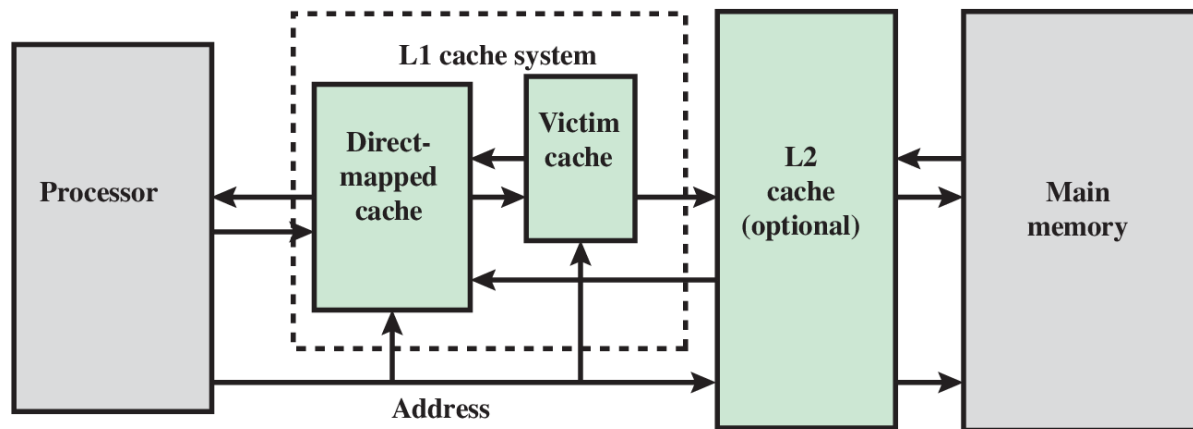


Figure D.1 Position of Victim Cache

Example:

A victim cache with 4 lines reduces conflict misses in a 4KB cache by 20% to 90%

A2: Pseudo-Associative Cache

Fast hit time of *direct mapped* and lower conflict misses of 2-way *set-associative* cache?

Divide cache: on a miss, check other half of cache to see if there, if so have a pseudo-hit (slow hit)



Drawback:

- CPU pipeline design is hard if hit takes 1 or 2 cycles
- Better for caches not tied directly to processor (L2)
- Used in MIPS R1000 L2 cache, similar in UltraSPARC

A2: Pseudo-Associative Cache

To determine where block is placed

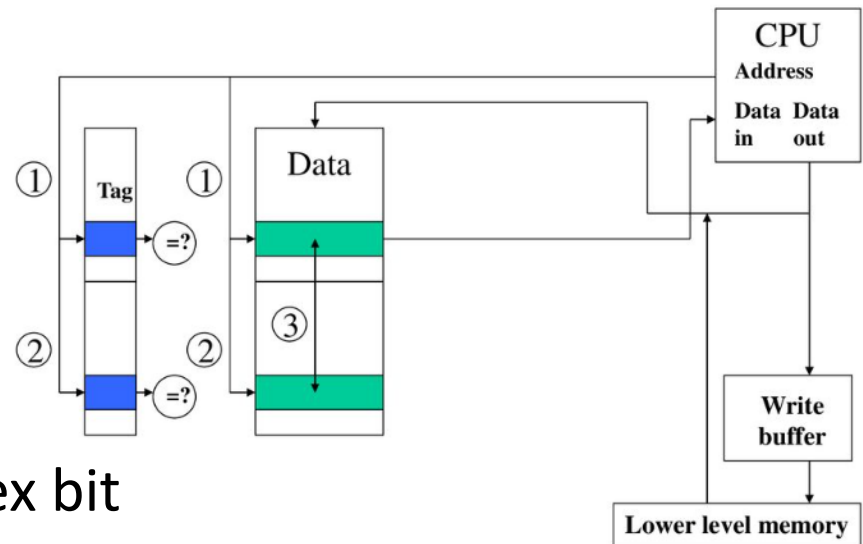
Check one block frame as in direct mapped cache, but If miss, check another block frame

E.g., frame with inverted MSB of index bit
Called a pseudo-set

Hit in first frame is fast

Placement of data

Put most often referenced data in “first” block frame and the other in the “second” frame of pseudo-set



Virtual Memory

User operates in a virtual address space, mapping between virtual space and main memory is determined at runtime

Original Motivation

- Avoid overlays

- Use main memory as a cache for disk

Current motivation

- Relocation

- Protection

- Sharing

- Fast startup

Engineered differently than CPU caches

- Miss access time $O(1,000,000)$

- Miss access time $\gg$ miss transfer time

Virtual Memory, cont.

Blocks, called *pages*, are 512 to 16K bytes.

Page placement

- Fully-associative -- avoid expensive misses

Page identification

- Address translation -- virtual to physical address

- Indirection through one or two page tables

- Translation cached in translation buffer

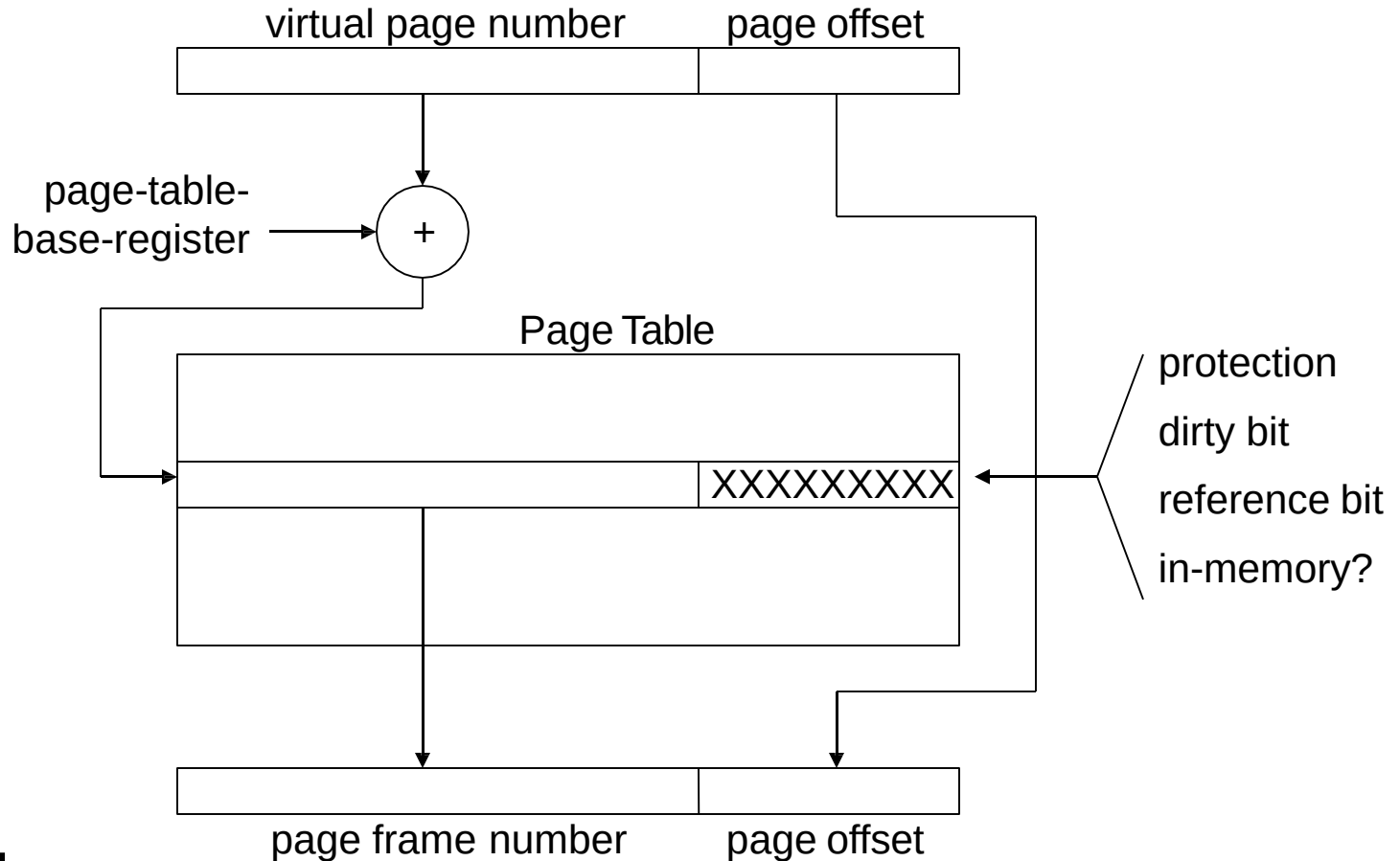
Page replacement

- Approx. LRU

Write strategy

- Writeback (with page dirty bit)

Address Translation



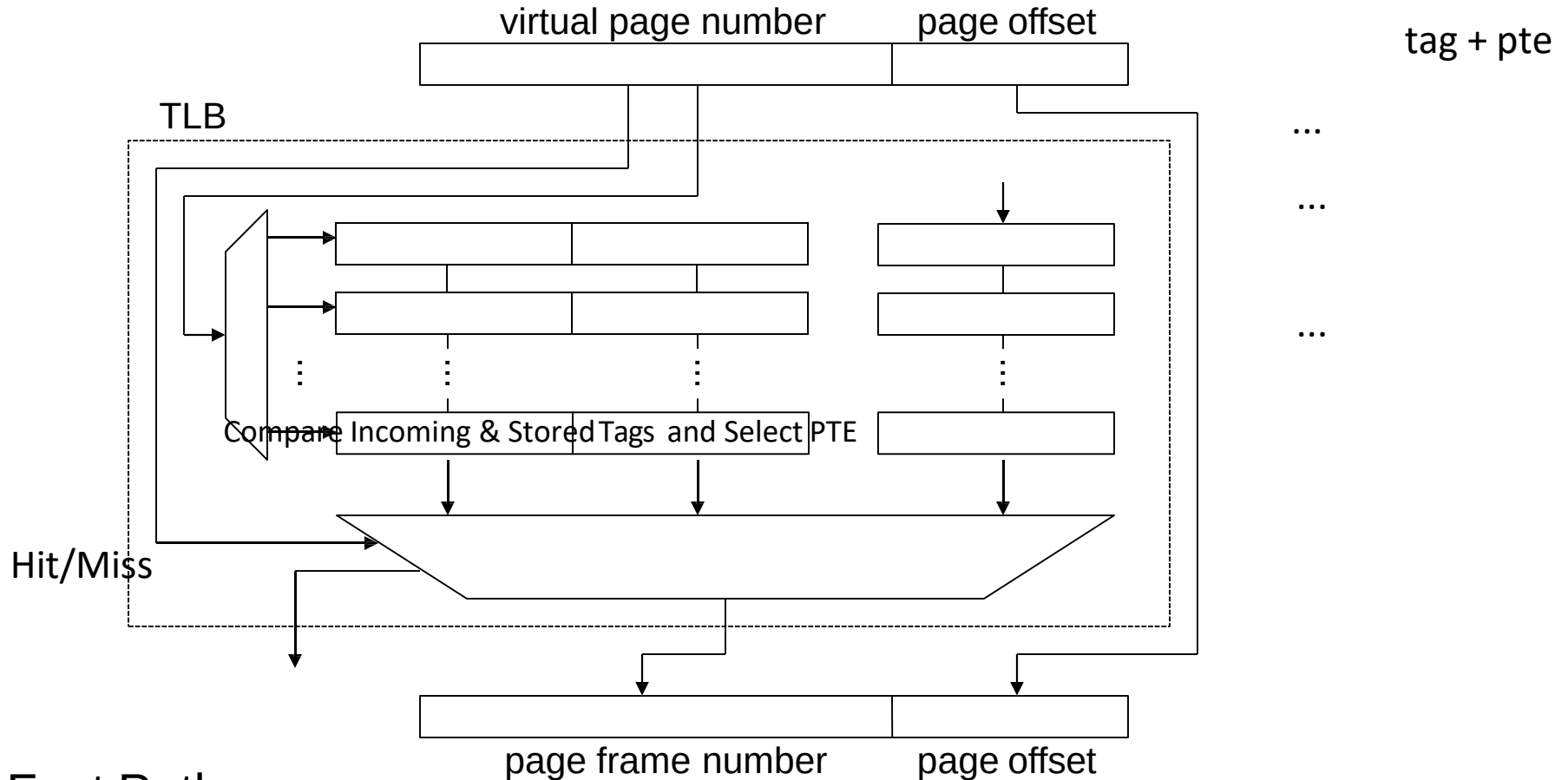
Logical Path

Two memory operations

Often two or three levels of page tables

TOO SLOW!

Address Translation



Fast Path

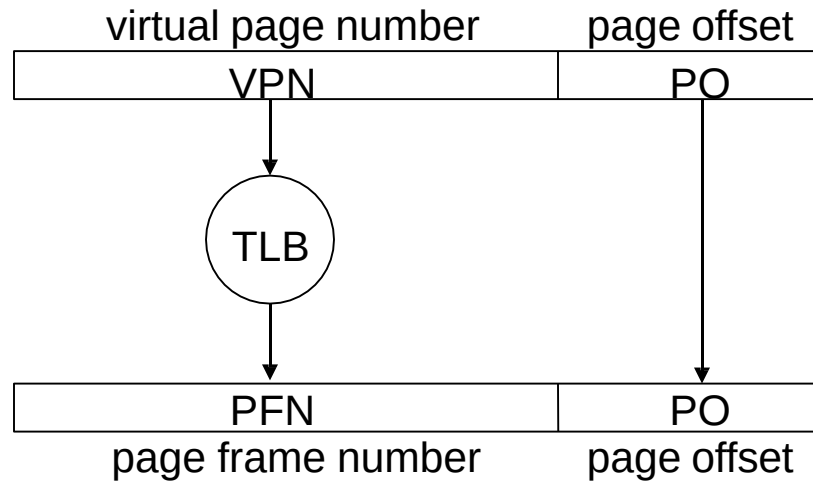
Translation Lookaside Buffer (TLB, TB)

A cache w/ PTEs for data

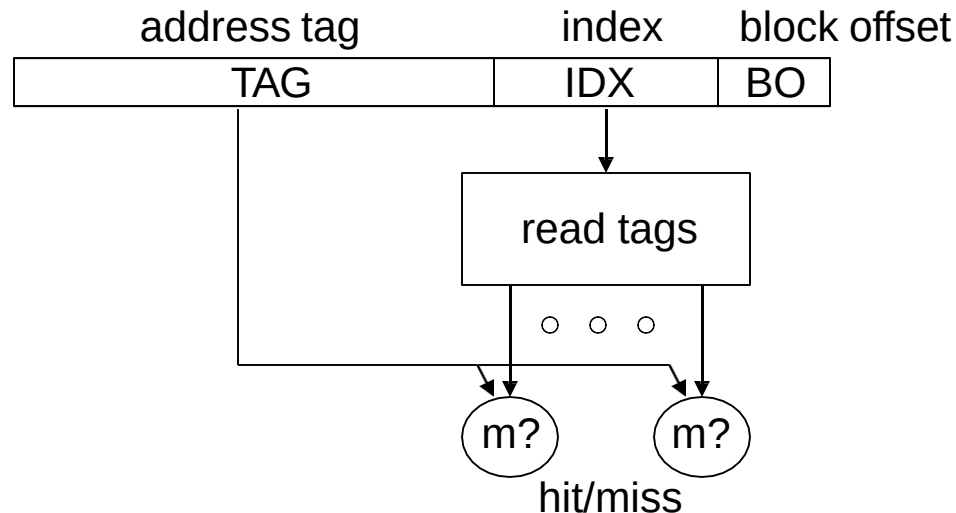
Number of entries 32 to 1024

Address Translation / Cache Interaction

Address Translation

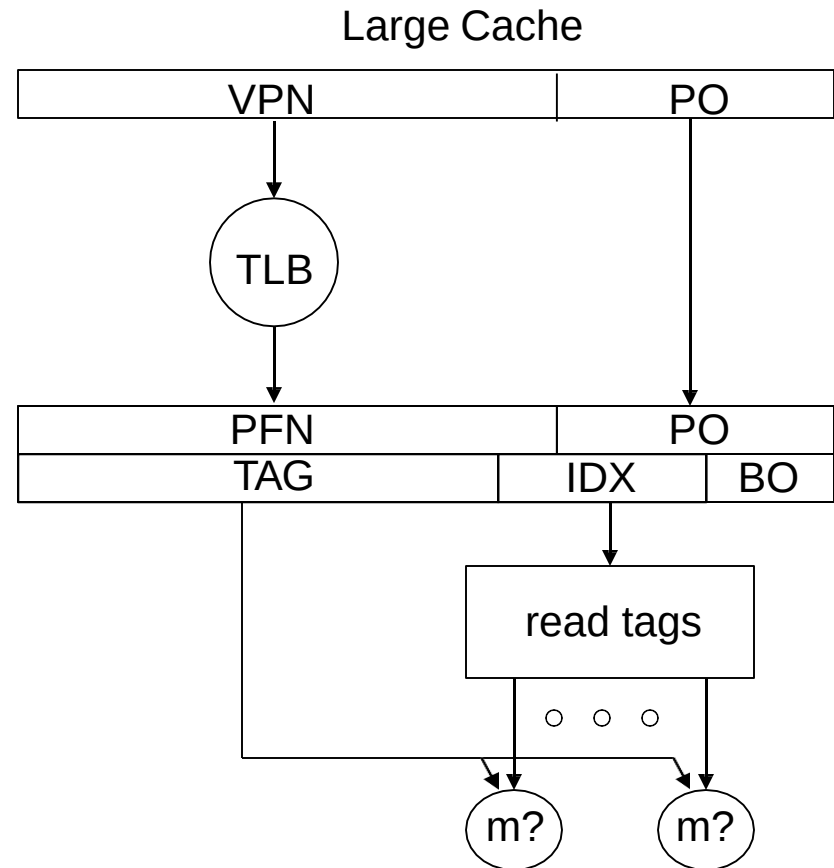
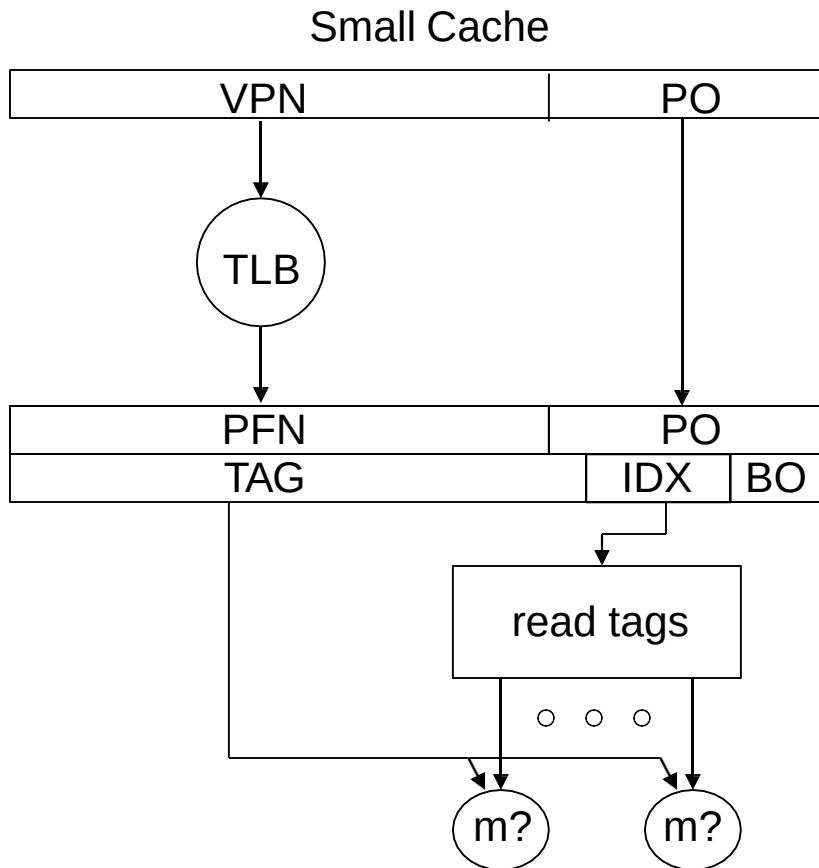


Cache Lookup



Sequential TLB Access

Address translation before cache lookup



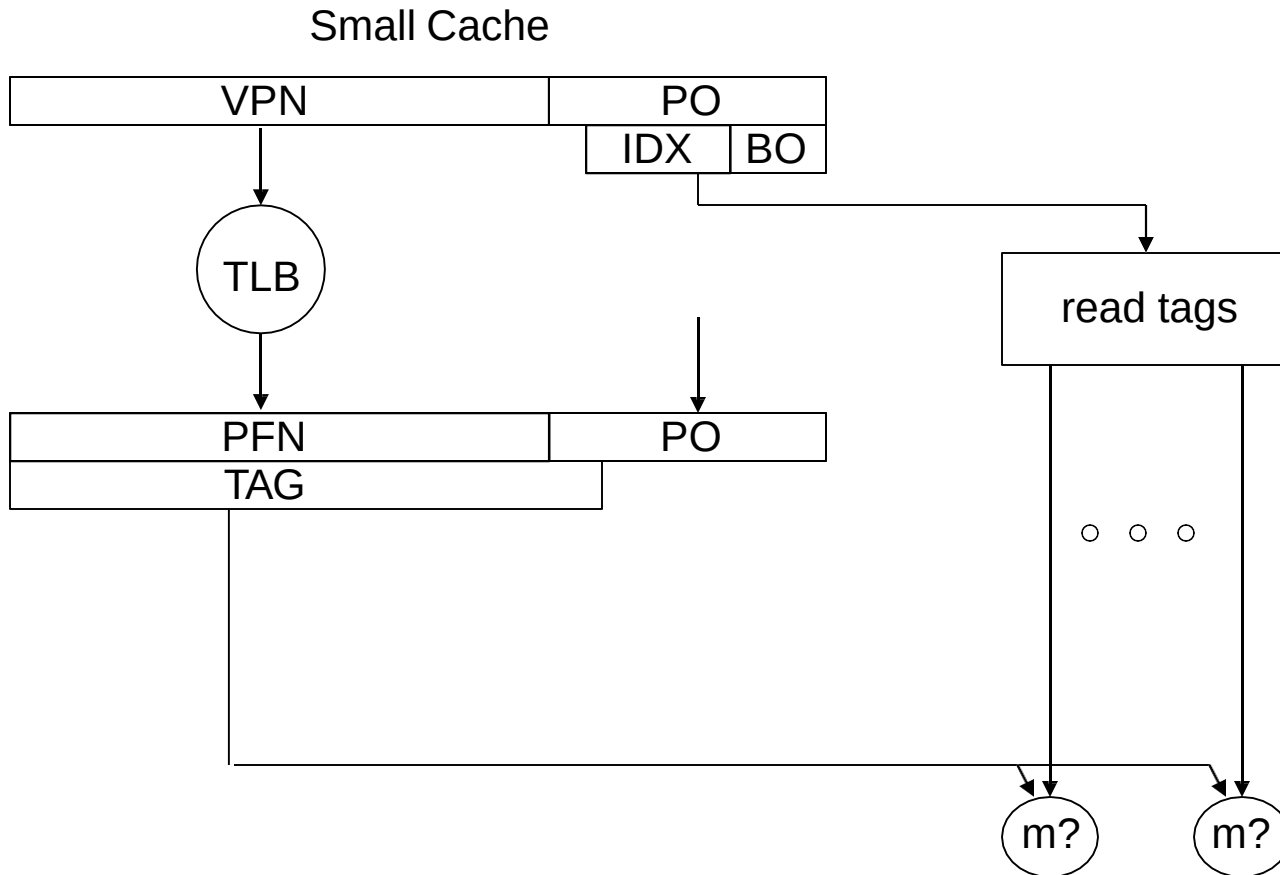
Problems

Slow

May increase cycle time, CPI, pipeline depth

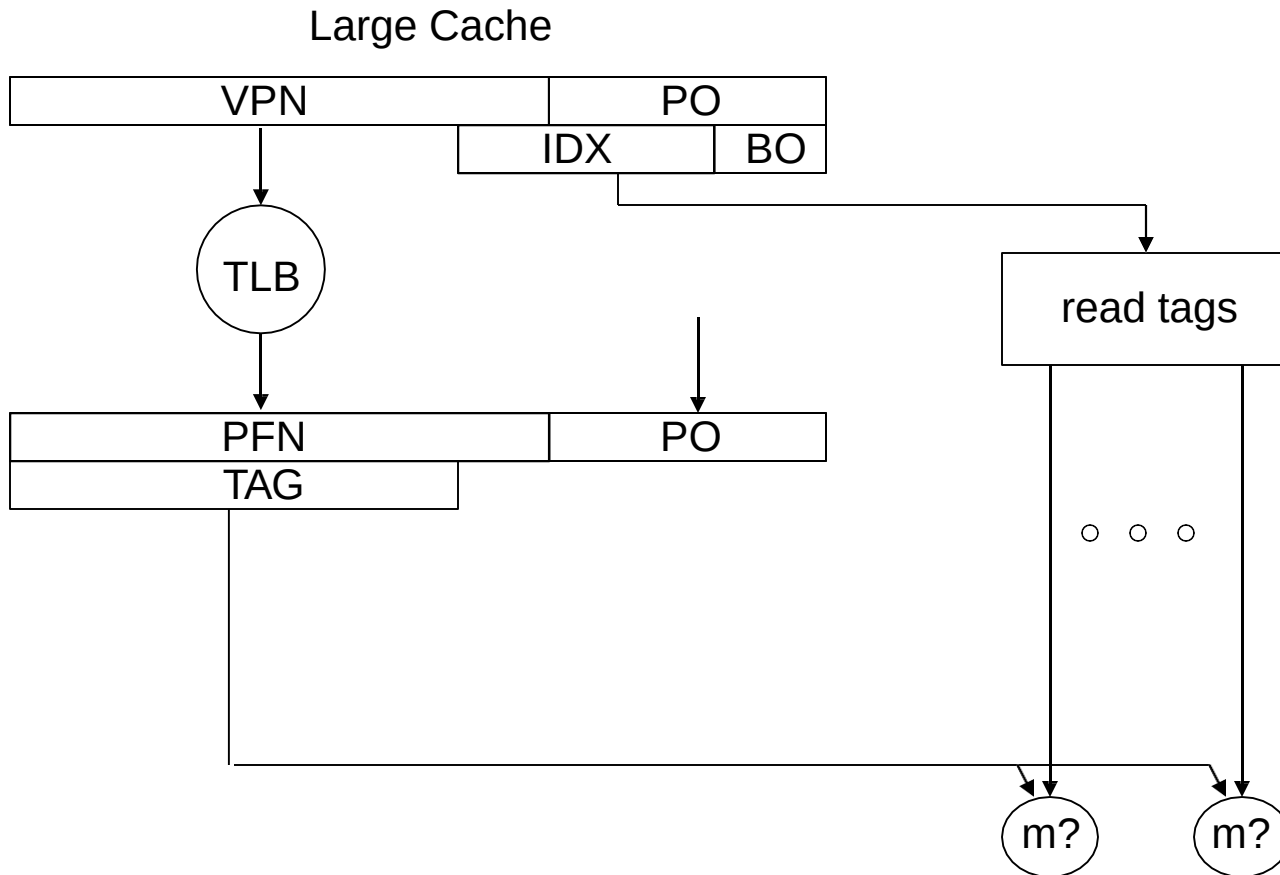
Parallel TLB Access

Address translation in parallel with cache lookup

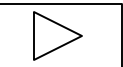


Parallel TLB Access

Address translation in parallel with cache lookup

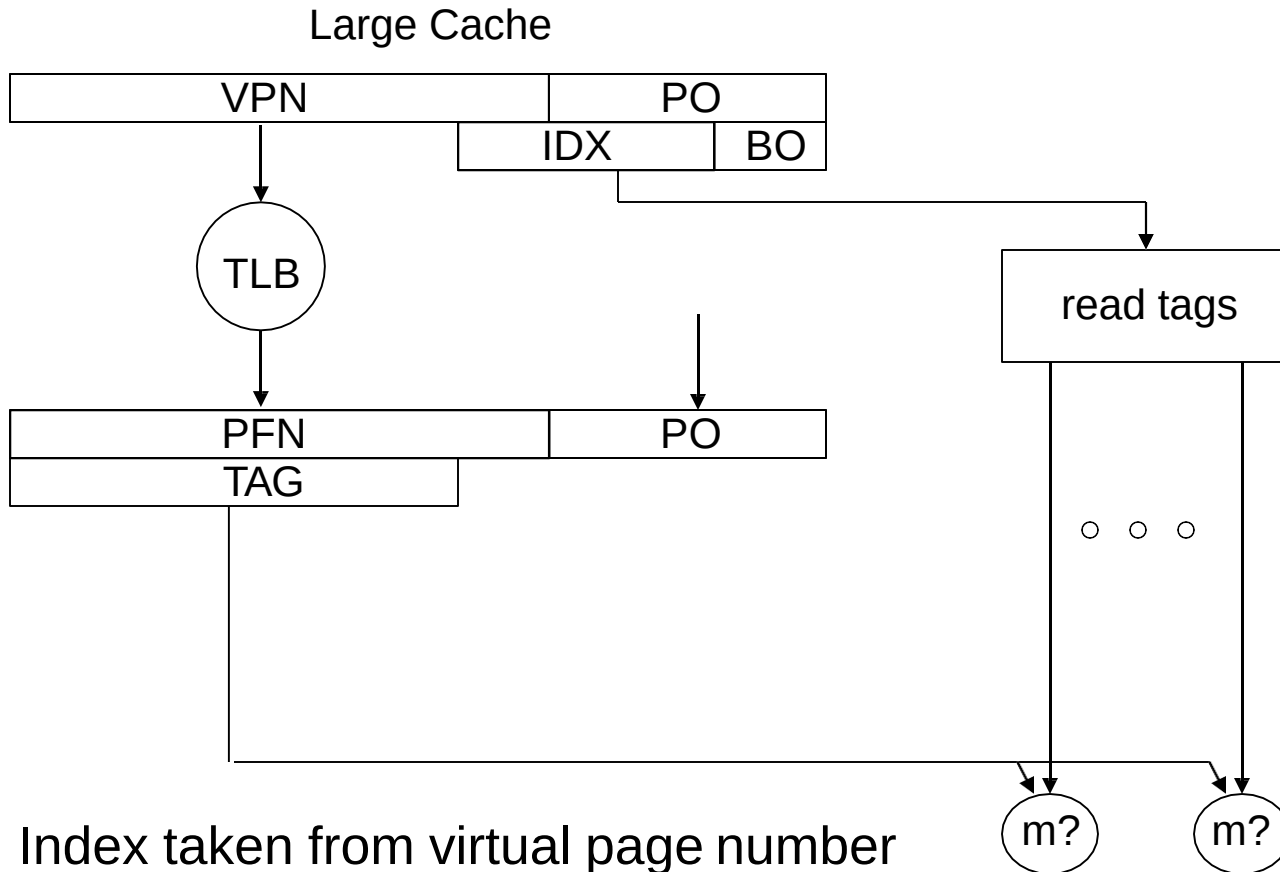


Index taken from virtual page number



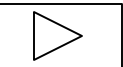
Parallel TLB Access**

Address translation in parallel with cache lookup



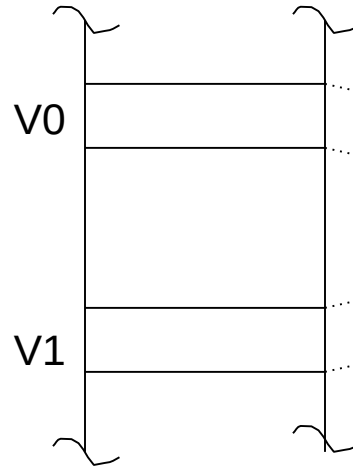
Index taken from virtual page number

Could cause problems with synonyms

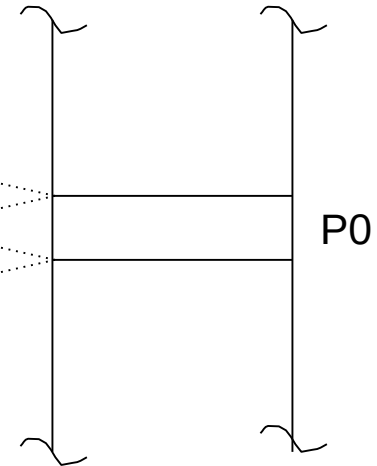


Virtual Address Synonyms

Virtual Address Space



Physical Address Space



	Tag	Data
Virtual Index		
	V0	
	V1	

Solutions to Synonyms

Solutions to Synonyms**

- (1) Limit cache size to page size times assoc
Extract index from page offset

Solutions to Synonyms**

- (1) Limit cache size to page size times assoc
Extract index from page offset
- (2) Search all sets in parallel
e.g., 64 KB 4way cache w/ 4KB pages
Search 4 sets (16 entries) in parallel

Solutions to Synonyms**

- (1) Limit cache size to page size times assoc
Extract index from page offset
- (2) Search all sets in parallel
e.g., 64 KB 4way cache w/ 4KB pages
Search 4 sets (16 entries) in parallel
- (3) Restrict page placement in operating system
Guarantee that $\text{Index(VA)} == \text{Index(PA)}$

Solutions to Synonyms**

- (1) Limit cache size to page size times assoc
Extract index from page offset
- (2) Search all sets in parallel
e.g., 64 KB 4way cache w/ 4KB pages
Search 4 sets (16 entries) in parallel
- (3) Restrict page placement in operating system
Guarantee that $\text{Index(VA)} == \text{Index(PA)}$
- (4) Eliminate by operating system convention
Single virtual address space
Restrictive sharing model

Virtual Address Cache



Address translation after cache miss

Implies fast lookup even for large caches

Must handle

Virtual address synonyms (aliases)

Virtual address space changes

Status and protection bit changes

Protection

Goal:

One process should not be able to interfere with the execution of another

Process model

Privileged kernel

Independent user processes

Primitives vs. Policy

Architecture provides the primitives

Operating system implements the policy

Problems arise when hardware implements policy

Protection Primitives

User vs. Kernel

- At least one privileged mode

- Usually implemented as mode bit(s)

How do we switch to kernel mode?

- Change mode and continue execution at *predetermined* location

Hardware to compare mode bits to access rights

- Access certain resources only in kernel mode

Protection Primitives, cont.

Base and Bounds

Privileged registers

$\text{Base} \leq \text{Address} \leq \text{Bounds}$

Pagelevel protection

Protection bits in page table entry

Cache them in TLB

Acknowledgements

These slides contain material developed and copyright by:

- Sarita Adve (UIUC)
- Nathan Beckmann (CMU)
- David Patterson (UC Berkeley)
- Xuehai Zhou (USTC)
- Qianni Deng (SJTU)
- Changsheng Xie (HUST)